

SYNTHETIX · SECURITY REVIEW

Synthetix TLX Review.

A security review of TLX Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	TLX Review	Guardian	December 3 – December 12, 2024	December 19, 2024

§ 00

Contents.

54 pages — overview, confidence ranking, scope, 39 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 34 findings	P. 10
	Remediation Review · 5 findings	P. 48
§ 05	Disclaimer	P. 53
§ 06	About Guardian	P. 54

§ 01

Overview.

Guardian reviewed TLX Review across 2 review rounds. The final report includes 3 High, 13 Medium, 23 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/tlxv3Audit
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
TLX Review	9d573b2160c7
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	6c89ebc66d64
AUDIT WINDOW	
December 3 – December 12, 2024	
FINAL REPORT	
December 19, 2024	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	3	0	0	1	2
Medium	13	0	9	0	4
Low	23	0	15	0	8
Informational	0	0	0	0	0
All severities	39	0	24	1	14

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed TLX Review in `Synthetixio/tlxv3Audit`. The scope includes 1,833 source lines across 14 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/ZapSwap.sol	171	251
src/TlxUpkeepRegistry.sol	51	72
src/RebalanceUpkeepSetupManager.sol	101	122
src/PythPriceHandler.sol	44	59
src/ProxyOwner.sol	100	132
src/ParameterProvider.sol	101	147
src/LeveragedTokenProxy.sol	12	18
src/LeveragedTokenFactory.sol	192	244
src/LeveragedToken.sol	445	587
src/AddressProvider.sol	136	205
src/upkeeps/RebalanceUpkeep.sol	99	125
src/upkeeps/LinkTopupUpkeep.sol	148	183
src/upkeeps/FundUpkeep.sol	194	242
src/upkeeps/BaseUpkeep.sol	39	57

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Rebalance Extractable Value	Sandwich Attack	HIGH	PARTIALLY RESOLVED	p. 10
H-02	Slippage Amounts Charged Are Lost	Logical Error	HIGH	RESOLVED	p. 11
M-01	Operations DoS When Minimum Credit Is Invalidated	DoS	MEDIUM	ACKNOWLEDGED	p. 13
M-02	Stale hasPendingLeverageUpdate	Validation	MEDIUM	RESOLVED	p. 14
M-03	Deposits Allowed While Liquidatable	Unexpected Behavior	MEDIUM	RESOLVED	p. 15
M-04	Depositors May Be Trapped Due To Pending PnL	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 16
M-05	Decaying Redemption Fee Manipulation	Gaming	MEDIUM	ACKNOWLEDGED	p. 17
M-06	Redeployed Token Lost From Token Mappings	Logical Error	MEDIUM	RESOLVED	p. 19
M-07	Outdated Price Used For Rebalances	Validation	MEDIUM	ACKNOWLEDGED	p. 20
M-08	Missing Referral Accrediting	Logical Error	MEDIUM	ACKNOWLEDGED	p. 21
M-09	Lacking Mint Referral	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 22
M-10	Stale Price Arbitrage	Gaming	MEDIUM	ACKNOWLEDGED	p. 23
M-11	Order Fees Do Not Include Settlement Costs	Logical Error	MEDIUM	ACKNOWLEDGED	p. 24

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-12	Max Market Size Unchecked	Logical Error	MEDIUM	RESOLVED	p. 25
L-01	Storage Compatibility	Best Practices	LOW	ACKNOWLEDGED	p. 26
L-02	Unexpectedly Large Redemption Fee	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 27
L-03	Price Impact Charged Successively	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 28
L-04	Unnecessary Return Value	Optimization	LOW	RESOLVED	p. 30
L-05	Small Actions Prevented By Price Impact	Warning	LOW	ACKNOWLEDGED	p. 31
L-06	Max Leverage Inaccuracy	Warning	LOW	ACKNOWLEDGED	p. 32
L-07	Lacking safeTransfer	Best Practices	LOW	RESOLVED	p. 34
L-08	Lacking Expo Validation	Validation	LOW	RESOLVED	p. 35
L-09	Lacking resetFailedCounter Function	Warning	LOW	RESOLVED	p. 36
L-10	Superfluous LinkTopupUpkeep Instances	Unexpected Behavior	LOW	RESOLVED	p. 37
L-11	Incorrect Interface Return Value	Logical Error	LOW	RESOLVED	p. 38
L-12	FundUpkeep Failure Due To Large Amounts	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 39
L-13	Base Amount Griefing	Warning	LOW	ACKNOWLEDGED	p. 40
L-14	Arbitrary Address Warning	Warning	LOW	ACKNOWLEDGED	p. 41
L-15	Hardcoded Positions	Warning	LOW	RESOLVED	p. 42
L-16	Users May Avoid Streaming Fee	Gaming	LOW	ACKNOWLEDGED	p. 43
L-17	Streaming Fee DoS	DoS	LOW	ACKNOWLEDGED	p. 44
L-18	Rebalances Can Occur While Paused	Warning	LOW	ACKNOWLEDGED	p. 45
L-19	Order Fee Charged May Not Cover Total Fee	Warning	LOW	ACKNOWLEDGED	p. 46

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-20	Leverage Not Reached Due To Pricing Discrepancies	Warning	LOW	ACKNOWLEDGED	p. 47

REMEDIATION REVIEW

H-01	Vault Drained By Cancellation Fees	Gaming	HIGH	RESOLVED	p. 48
M-01	Only Current Mint Amount Validated	Validation	MEDIUM	ACKNOWLEDGED	p. 49
L-01	Missing Buffer In Size Validation	Validation	LOW	RESOLVED	p. 50
L-02	Lacking safeTransfer	Best Practices	LOW	ACKNOWLEDGED	p. 51
L-03	Outstanding TODOs	Warning	LOW	ACKNOWLEDGED	p. 52

HIGH **H-01**

PARTIALLY RESOLVED

Rebalance Extractable Value

CATEGORY	Sandwich Attack
LOCATION	LeveragedToken.sol
REVIEW	Main Review

DESCRIPTION

In the `_submitLeverageUpdate` function the `acceptablePrice` value is determined by applying a standard slippage amount to the result of the `fillPrice` function.

However the result of the `fillPrice` function itself can be manipulated such that it returns a higher `fillPrice` and thus allows for significant extractable value by sandwiching the TLX vault's order.

Consider the following scenario:

- A malicious actor observes that a significant amount of PnL has built up for the TLX vault and that a rebalance will be triggered by even a small deposit.
- The malicious actor creates a large long order to push the skew of the market higher.
- The malicious actor triggers a small deposit with the `mintFor` function, triggering a rebalance.
- The rebalance order is assigned a high `acceptablePrice` which can be significantly more than the fair market value of the index asset due to the inflated price impact.
- The malicious actor subsequently closes their position directly after the rebalance order is executed, receiving positive impact at the expense of the leveraged token vault holders.

RECOMMENDATION

Consider basing the acceptable price off of an independent oracle which cannot be manipulated on-chain.

RESOLUTION

Status: Partially resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/64>

HIGH **H-02**

RESOLVED

Slippage Amounts Charged Are Lost

CATEGORY Logical Error
LOCATION LeveragedToken.sol: 162
REVIEW Main Review

DESCRIPTION

In the `redeemFor` function the `baseAmountReceived` which is sent to the user is reduced by the `slippage` amount, however the full `baseWithdrawn` amount which is removed from the margin in Perps V3 includes this slippage amount.

Therefore the slippage amount of sUSD will sit in the `LeveragedToken` contract and will now not be tracked as value in the system since the `totalValue` result is entirely dependent on the result of `perpsMarket.getAvailableMargin`.

Explicitly, the accounting which occurs on redemption is as follows.

Removed from SNX and credited to the `LeveragedToken` contract:

- `baseWithdrawn`

Debited from the `LeveragedToken` contract and sent elsewhere:

- `baseAmountReceived = baseWithdrawn - fee - slippage`
- `fee`

Thus it is clear to see that `baseWithdrawn` is received and `baseAmountReceived + fee = baseWithdrawn - fee - slippage + fee = baseWithdrawn - slippage` is sent out. This results in `slippage` amount of sUSD being left in the contract.

RECOMMENDATION

Deduct `slippage` from `baseWithdrawn` after computing the slippage. Notice that this will charge the `feePercent` on the amount which will be withdrawn and sent to the user instead of the amount to be withdrawn plus the slippage. We believe this behavior to be correct, but the protocol should independently verify this.

RESOLUTION

Status: Resolved.

H-02 Slippage Amounts Charged Are Lost

CONTINUED

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/62>

MEDIUM **M-01**

ACKNOWLEDGED

Operations DoS When Minimum Credit Is Invalidated

CATEGORY DoS
LOCATION LeveragedToken.sol
REVIEW Main Review

— DESCRIPTION

In the `_validateMintAmount` function the mint amount is validated to be within the maximum open interest for the relevant side and market. However the Perps market as a whole is not validated to be within the `minimumCredit` constraint.

The deposited funds may put the Perps market over the allowed open interest based on the current credit capacity delegated to it by the V3 core system. This results in a DoS of all deposit/withdrawal actions as they attempt to commit orders which will revert due to the minimum credit validation here:

<https://github.com/Synthetixio/synthetix-v3/blob/306c928de0da52d6e839f0620f8f32d544217648/markets/perps-market/contracts/storage/AsyncOrder.sol#L371>

— RECOMMENDATION

Include the minimum credit validation with an appropriate buffer in the `_validateMintAmount` function.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Stale hasPendingLeverageUpdate

CATEGORY Validation
LOCATION LeveragedToken.sol: 314
REVIEW Main Review

DESCRIPTION

The `hasPendingLeverageUpdate` function relies upon the `sizeDelta` of the stored order in the Perps V3 system, however stale orders which will be automatically overwritten upon a new order commitment still exist in the order storage.

Therefore stale orders are considered a pending leverage update even though they are inactive. In the case that a stale order is left for the system this will prevent rebalances during this period of time.

RECOMMENDATION

Consider checking if the current timestamp is within the `settlementStrategy.settlementDelay + settlementStrategy.settlementWindowDuration` of the order commitment, if it is not then the order is stale and should not be considered pending.

RESOLUTION

Status: Resolved.

MEDIUM **M-03**

RESOLVED

Deposits Allowed While Liquidatable

CATEGORY Unexpected Behavior
LOCATION LeveragedToken.sol: 77
REVIEW Main Review

DESCRIPTION

In the `mintFor` function there is no validation that deposits cannot occur while the leveraged token's Perps V3 position is in a liquidatable state. If a rebalance is triggered by the deposit then the validation in the SNX `validateRequest` function will prevent such an action. However if the deposit is not large enough, or if the deposit corrects for a high leverage caused by losses and thus puts the leverage back in range and does not trigger a rebalance then the deposit will be allowed while the position is liquidatable.

Users who deposit in this case will experience an immediate loss of funds as there is no way to withdraw their assets due to the liquidation checks and the position will be subsequently liquidated.

RECOMMENDATION

Consider validating that the position is not in a liquidatable state in the `_validateMintAmount` function. Notice that being in a liquidatable state is not the same as checking if the position is flagged for liquidation, the `LiquidationModule.canLiquidate` function should be used for a complete validation.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/63>

MEDIUM **M-04**

ACKNOWLEDGED

Depositors May Be Trapped Due To Pending PnL

CATEGORY Unexpected Behavior
LOCATION LeveragedToken.sol
REVIEW Main Review

— DESCRIPTION

In Synthetix V3 pending profits cannot be immediately withdrawn from the system until an order has been made to settle those profits to the account margin.

This behavior means that the TLX system may be illiquid for users to withdraw from until a rebalance occurs. In the past TLX has recommended a recursive approach for withdrawals to trigger a rebalance so that the pending PnL can be settled to margin in order to withdraw.

However there are certain edge cases that this approach would miss. The clearest example is if the Perps V3 position is in a state where there is 0 margin and all pending profit. In this case no withdrawal can be triggered to induce a rebalance and unlock all of the pending PnL as liquidity for withdrawers.

This scenario can be achieved if smaller withdrawals are made which stay within the deviation range and these withdrawals are subsequently corrected for in the PnL changes.

— RECOMMENDATION

This is a rare edge case and may never present itself in production. Nevertheless it may be helpful to include a trusted function to settle outstanding PnL by submitting a dummy order to the Perps V3 system so that users can get liquidity to withdraw.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Decaying Redemption Fee Manipulation

CATEGORY Gaming
LOCATION LeveragedToken.sol
REVIEW Main Review

— DESCRIPTION

In the leveraged token system a decaying redemption fee is applied to users who have recently deposited. Throughout the codebase the decaying redemption fee is assigned to start off at a 25% fee and decay to 0% over the course of 25 minutes.

The minimum deposit for a user which will reset the timer for the decaying redemption fee is set as $1e18$ or 1 USD in the `Config` contract as the `DECAYING_REDEMPTION_MIN_BASE_AMOUNT` value.

With these configured parameters it can be significantly profitable for one vault depositor to do a small deposit on behalf of another depositor who is about to withdraw and cause them to experience a significant decay fee. The malicious vault depositor in this case would gain from the significant fee paid by the victim depositor in this case.

On networks without a public mempool specifically frontrunning a user's withdrawal transaction is not reliably possible so this attack may operate based upon key indicators that a user is about to withdraw such as Discord messages or market volatility.

Furthermore, if the `DECAYING_REDEMPTION_MIN_BASE_AMOUNT` is configured too high, then a depositor could simply deposit `DECAYING_REDEMPTION_MIN_BASE_AMOUNT - 1` wei multiple times to avoid the decaying redemption fee while still depositing a large amount. This could occur in a single transaction with a multicall or for-loop contract call around the `mintFor` function.

— RECOMMENDATION

As a first option, consider requiring that a user has approved another user to `mintFor` on their behalf.

Alternatively, configure the `DECAYING_REDEMPTION_MIN_BASE_AMOUNT`, `decayingRedemptionFeeStart`, and `decayingRedemptionFeeDuration` with these behaviors in mind. Ensuring that the `DECAYING_REDEMPTION_MIN_BASE_AMOUNT` is neither too low to incentivize bad faith mints on behalf of other users and that the `DECAYING_REDEMPTION_MIN_BASE_AMOUNT` value is not too high to incentivize split deposits to avoid the decay fee measure.

M-05 Decaying Redemption Fee Manipulation

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-06**

RESOLVED

Redeployed Token Lost From Token Mappings

CATEGORY Logical Error
LOCATION LeveragedTokenFactory.sol
REVIEW Main Review

DESCRIPTION

Function `redeployInactiveToken` calls `delete _tokens[marketId][targetLeverage][isLong]` which deletes the newly deployed token rather than the old, inactive token. Consequently `tokenExists` will produce an incorrect result and leveraged tokens can be created even for active markets with `createLeveragedTokens` since `if (tokenExists(marketId, targetLeverage, true))` passes.

RECOMMENDATION

Do not clear the mapping, since it is set to the new token upon redeploy.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/65>

MEDIUM **M-07**

ACKNOWLEDGED

Outdated Price Used For Rebalances

CATEGORY Validation
LOCATION LeveragedToken.sol: 194
REVIEW Main Review

DESCRIPTION

The `rebalance` function does not include any validation against the oracle price staleness with the `_ensureOraclePriceLiveliness` function, nor does it offer a way for the upkeep caller to update the pyth price.

Therefore stale prices may be used on accident or on purpose which will incorrectly compute the `sizeDelta` for the leverage update of the system. This will ultimately perturb the leverage of the vault system after a rebalance because the price used to determine the `sizeDelta` can be in some cases on the order of days old.

RECOMMENDATION

Include a `_ensureOraclePriceLiveliness` validation in the `rebalance` function as well as a pathway to update the oracle price with the `_updateOraclePrice` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Missing Referral Accrediting

CATEGORY Logical Error
LOCATION LeveragedToken.sol: 397
REVIEW Main Review

DESCRIPTION

In the `_chargeRedemptionFee` function if a `referralCode` is provided the `baseAsset` is transferred to the `referrals` address, however there is no logic to accredit the referral fee amount to the referrer who owns the `referralCode`.

RECOMMENDATION

Implement the necessary logic to accredit the referrer who owns the `referralCode`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Lacking Mint Referral

CATEGORY Unexpected Behavior
LOCATION LeveragedToken.sol: 77
REVIEW Main Review

DESCRIPTION

The `mintFor` function accepts a `referralCode` parameter, however no referral fee is charged. Currently the `referralCode` is only emitted in the `Minted` event.

RECOMMENDATION

Implement the referral fee logic for the `mintFor` function, otherwise remove the `referralCode` parameter.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

Stale Price Arbitrage

CATEGORY Gaming
LOCATION LeveragedToken.sol
REVIEW Main Review

DESCRIPTION

In the `LeveragedToken` contract the pyth feed used to determine the value of the system position is required to be updated with a price in the last 60 seconds of the action being performed.

In times of volatility this can present an arbitrage opportunity whereby a price from 60 seconds ago is abused to make a net positive deposit/withdrawal pair from the vault.

Consider the following scenario:

- The leveraged token vault is long
- Price increases by 1% for the index asset of the vault in the last 60 seconds, from \$100 to \$101
- User A has an existing deposit of 10 tokens in the vault
- User A makes a deposit for 10 tokens with a new address and provides pyth update data for 60 seconds ago, when the price is \$100
- Directly after this User A makes a withdrawal for their original 10 token address and provides the latest Pyth data of \$101

User A has now arbitrated the difference between the \$100 price and \$101 price. Having withdrawn at \$101 and deposited the same amount at \$100.

RECOMMENDATION

Consider reducing the staleness period allowed for the Pyth feed. Furthermore, consider adding an additional fee that is charged to the user upon deposit/withdrawal to reduce the feasibility of this attack.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Order Fees Do Not Include Settlement Costs

CATEGORY Logical Error
LOCATION LeveragedToken.sol: 612
REVIEW Main Review

— DESCRIPTION

In the `_orderFee` function the order fee computed does not include the settlement costs which are levied based on the `settlementRewardCost` in the Perps V3 system.

This is a one time hard cost that will be applied to every rebalance that occurs and is not specifically remunerated by the depositors/withdrawers who are triggering the rebalance.

— RECOMMENDATION

Consider if this is acceptable. If it is not, consider requiring that the actor who triggers the rebalance covers the fee for the settlement cost.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

RESOLVED

Max Market Size Unchecked

CATEGORY Logical Error
LOCATION LeveragedToken.sol: 507
REVIEW Main Review

DESCRIPTION

The Perps market `validatePositionSize` validation validates both the max market size and max market value, however the TLX `_validateMintAmount` validation only validates the max market value.

It is possible that the max value validation is not reached but the max size validation is reached, thus allowing a mint to occur in the TLX vault when the corresponding order cannot go through causing a DoS for actions that would trigger a rebalance.

RECOMMENDATION

Consider adding validation for the market size in addition to the existing value validation.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/72>

LOW **L-01**

ACKNOWLEDGED

Storage Compatibility

CATEGORY	Best Practices
LOCATION	TlxOwnableUpgradeable.sol
REVIEW	Main Review

— DESCRIPTION

The TlxOwnableUpgradeable contract is intended to be compatible with upgradeable contracts, but it does not use namespaced storage.

This can create issues with future upgrades if the storage layout were to change as documented here: <https://eips.ethereum.org/EIPS/eip-7201>.

— RECOMMENDATION

Consider using namespaced storage in the `TlxOwnableUpgradeable` contract.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Unexpectedly Large Redemption Fee

CATEGORY Unexpected Behavior
LOCATION LeveragedToken.sol: 161
REVIEW Main Review

DESCRIPTION

In the `redeemFor` function the redemption fee is multiplied by the target leverage to determine the fee amount levied on the redeemer. The redemption fee percentage is currently configured as 0.5% in the `Config` contract, therefore if the leveraged token vault is using a target leverage of 10x the redemption fee can be as high as 5%.

This value seems quite high for a redemption fee and may be unexpected by the protocol.

RECOMMENDATION

Confirm whether the redemption fee ought to be multiplied by the target leverage, effectively making it a proxy for a notional value fee. If the redemption fee is not meant to reach 5% of the redeemers deposited funds then consider removing the leverage multiplication.

RESOLUTION

Status: Acknowledged.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/70>

LOW **L-03**

ACKNOWLEDGED

Price Impact Charged Successively

CATEGORY Unexpected Behavior
LOCATION LeveragedToken.sol: 295
REVIEW Main Review

DESCRIPTION

In the `computePriceImpact` function the price impact for each deposit/withdrawal is computed regardless of if it triggers a rebalance. Furthermore the price impact charged to the user is not concerning the individual contribution the user made to the order size, but instead the entire price impact experienced by the whole `rebalanceSizeDelta`.

Consider the following scenario in which the `rebalanceSizeDelta` which will correspond to a rebalance is 100 index tokens.

- `rebalanceSizeDelta` is initially 0
- Leverage is 2x for example
- 1 sUSD == 1 index token
- User A deposits 20 sUSD and is charged impact for a `rebalanceSizeDelta` of 40
- User A deposits 20 more sUSD and is charged impact for a `rebalanceSizeDelta` of 80
- User A deposits 20 more sUSD and is charged impact for a `rebalanceSizeDelta` of 120
- The `rebalanceSizeDelta` is now greater than 100 and thus an order is triggered. The vault pays impact for an order size of 120. However user A has paid impact fees for each of 40, 80, and 120 sizeDelta orders.

RECOMMENDATION

Consider if this behavior is acceptable. If it is not then consider one of the following solutions.

Firstly, the price impact on the `rebalanceSizeDelta` can only be levied as a fee to the user if `canRebalance` is true and the rebalance order will be submitted.

Alternatively, the price impact charged to the user can be the difference in price impact experienced between the pre-existing `rebalanceSizeDelta` which results from the marginBalance without their deposit/withdrawal and the new `rebalanceSizeDelta` which includes their deposit/withdrawal. In other words, the price impact that is charged to the user is exactly the price impact difference that their action induces. Notice that these two solutions should not be combined.

L-03 Price Impact Charged SuccessivelyCONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/66>

LOW **L-04**

RESOLVED

Unnecessary Return Value

CATEGORY Optimization
LOCATION LeveragedToken.sol: 281
REVIEW Main Review

DESCRIPTION

The `computePriceImpact` function returns a boolean as it's second return value, however this value is always assigned as true.

RECOMMENDATION

Consider removing the boolean from the return values of the `computePriceImpact` function or implement it's use-case.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/59>

LOW **L-05**

ACKNOWLEDGED

Small Actions Prevented By Price Impact

CATEGORY Warning
LOCATION LeveragedToken.sol: 101, 163
REVIEW Main Review

— DESCRIPTION

When performing a deposit or withdrawal action the price impact that would be experienced by the entire rebalance order is levied against the user. In some cases where there is a large pending rebalance and the deposit or withdrawal is relatively small this may result in an underflow revert when deducting the slippage from the amount to be deposited or withdrawn.

— RECOMMENDATION

Consider if this is acceptable behavior. If it is then be sure to document it for users and consider introducing a more descriptive custom error and validation for this case.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Max Leverage Inaccuracy

CATEGORY Warning
LOCATION LeveragedTokenFactory.sol: 262
REVIEW Main Review

DESCRIPTION

The `_maxLeverage` function computes the maximum allowable leverage as `1/minimumInitialMarginRatioD18_` however in the Perps V3 system the maximum allowed leverage for the initial margin validation is based on `minimumInitialMarginRatioD18 + initialMarginRatioD18 * impact0nSkew`.

By this logic the max leverage should be `1/(minimumInitialMarginRatioD18 + initialMarginRatioD18 * impact0nSkew)` where the `impact0nSkew` is based on the upper bound of leveraged token vault size.

In the following case we can examine how much the maximum leverage calculation would be off for the ETH market:

- `skewScale = 1_000_000e18`
- `initialMarginRatioD18 = 1160000000000000000 (1.16)`
- `minimumInitialMarginRatioD18 = 200000000000000000 (0.02)`
- Max expected vault size: 2,000 ETH

$$\text{maximum leverage} = 1 / (\text{minimumInitialMarginRatioD18} + \text{initialMarginRatioD18} * \text{impact0nSkew}) = 1 / (0.02e18 + 1.16e18 * 2_000e18/1_000_000e18) = 44.8028673835x \text{ max leverage}$$

However the existing calculation computes the maximum leverage as `1 / minimumInitialMarginRatioD18` = `1e18 / 0.02e18 = 50x max leverage`

RECOMMENDATION

The max leverage which is used to validate the initial leverage is simply divided by 2 to give a rough validation so that the target leverage allows some buffer between the maximum allowable initial margin, so some inaccuracy may be acceptable here.

If the level of inaccuracy demonstrated is not acceptable, then consider adopting the more exact max leverage calculation which incorporates the `initialMarginRatioD18` and `impact0nSkew`.

L-06 Max Leverage Inaccuracy

CONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

RESOLVED

Lacking safeTransfer

CATEGORY Best Practices

LOCATION ZapSwap.sol: 55

REVIEW Main Review

DESCRIPTION

In the `ZapSwap` mint function `transferFrom` is used on an arbitrary `zapAsset` without checking the return value.

RECOMMENDATION

To be compatible with all tokens use `safeTransferFrom` in case any token which does not revert on failure is supported.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/67>

LOW **L-08**

RESOLVED

Lacking Expo Validation

CATEGORY	Validation
LOCATION	PythPriceHandler.sol: 38
REVIEW	Main Review

— DESCRIPTION

In the PythPriceHandler contract there is no validation to ensure that the `pythPrice.expo` is indeed a negative number.

Currently there are no feeds which have this characteristic, however out an abundance of caution if any feeds were to be supported with a positive expo or if any feeds were to be misconfigured the application should revert instead of providing a significantly incorrect price result.

— RECOMMENDATION

Consider adding a validation against a positive `pythPrice.expo` value in the `getLatestPrice` function.

— RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/68>

LOW **L-09**

RESOLVED

Lacking resetFailedCounter Function

CATEGORY	Warning
LOCATION	FundUpkeep.sol
REVIEW	Main Review

DESCRIPTION

The `FundUpkeep` contract does not include a `resetFailedCounter` function which is seen in the `RebalanceUpkeep` contract. This function may be useful if any unexpected issues were to occur and the failed counter needed to be reset.

RECOMMENDATION

Consider adding a `resetFailedCounter` to the `FundUpkeep` contract in case of any unexpected scenarios where the failed counter should be reset.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/75>

LOW **L-10**

RESOLVED

Superfluous LinkTopupUpkeep Instances

CATEGORY	Unexpected Behavior
LOCATION	RebalanceUpkeepSetupManager.sol
REVIEW	Main Review

DESCRIPTION

The `setupRebalanceUpkeep` function configures a new `linkTopupUpkeep` for every new `marketId` which is configured. The `checkData` for this upkeep is the `marketId`, however the `checkUpkeep` function for the `LinkTopupUpkeep` contract does not make use of this upkeep check data.

Every instance of the `LinkTopupUpkeep` upkeep will check all upkeep contracts registered in the upkeep registry for upkeeps, therefore there is no need to configure a `LinkTopupUpkeep` for every market id.

RECOMMENDATION

Only configure the `LinkTopupUpkeep` once as this single instance can cover all upkeep contracts.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/71>

LOW **L-11**

RESOLVED

Incorrect Interface Return Value

CATEGORY Logical Error
LOCATION AddressProvider.sol: 198
REVIEW Main Review

DESCRIPTION

In the `linkTopupUpkeep` function the returned value uses the `IFundUpkeep` interface, however this is not the interface of the `linkTopupUpkeep` contract.

RECOMMENDATION

Correct the return value type to the `ILinkTopupUpkeep` interface and consider if there should be a function in the `AddressProvider` to retrieve the `FundUpkeep` contract instance.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/71>

LOW **L-12**

ACKNOWLEDGED

FundUpkeep Failure Due To Large Amounts

CATEGORY	Unexpected Behavior
LOCATION	FundUpkeep.sol
REVIEW	Main Review

DESCRIPTION

In the `FundUpkeep` contract the upkeep action attempts to balance the remaining margin value of the leveraged token at the `targetAmountPerToken` when the amount goes out of the minimum or maximum ranges.

However in the case where the deposits in a `LeveragedToken` vault become too large it may be impossible for the `FundUpkeep` contract to redeem to meet the `targetAmountPerToken` as it does not have enough leveraged tokens to do so. This will result in consistent failures for the upkeep action as the redeem action will continue to revert with an ERC20 balance underflow.

The same could occur for a mint upkeep action though is less likely as there is a lower bound of value which can be in the vault.

RECOMMENDATION

Be aware of this behavior and consider limiting the minted/redeemed amount to the balance of the `FundUpkeep` contract in these cases.

RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Base Amount Griefing

CATEGORY	Warning
LOCATION	ZapSwap.sol
REVIEW	Main Review

— DESCRIPTION

Because ZapSwap simply uses the entire balance of `baseAsset` within the contract to mint leveraged tokens, a malicious user can donate `baseAsset` tokens directly to the contract, and potentially trigger `MaxMarketValueExceeded` in the mint which would prevent users from utilizing the zap functionality.

— RECOMMENDATION

Either use the amount of bases assets obtained from the swap or clearly document this behavior.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Arbitrary Address Warning

CATEGORY	Warning
LOCATION	ZapSwap.sol
REVIEW	Main Review

— DESCRIPTION

The `zapAssetAddress` is an arbitrary address passed by the user which may lead to an unsafe, external contract taking control of execution.

— RECOMMENDATION

It is generally best practice to avoid arbitrary addresses. Consider validation on the `zapAssetAddress` or document this behavior.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

RESOLVED

Hardcoded Positions

CATEGORY	Warning
LOCATION	ZapSwap.sol
REVIEW	Main Review

DESCRIPTION

`_validate0dosSwapAllData` uses bytes 28 and 8 depending on whether the address is encoded or not to ensure the entire balance is being swapped. This works for `swapCompact` but not `swapMultiCompact` which loops through multiple inputs.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/75>

LOW **L-16**

ACKNOWLEDGED

Users May Avoid Streaming Fee

CATEGORY Gaming
LOCATION LeveragedToken.sol: 377
REVIEW Main Review

— DESCRIPTION

Users may observe that a significant streaming fee has accrued over time and choose to withdraw their deposits to avoid the streaming fee. So long as a user does not withdraw enough funds to trigger a rebalance, they will successfully avoid the streaming fee.

— RECOMMENDATION

Be aware of this minor gaming of the streaming fee. Know that the amount that can be avoided is limited to what the deviation factor will allow to go without rebalance. However if a solution is desired then the streaming fee can be levied on all deposits and withdrawals.

— RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

Streaming Fee DoS

CATEGORY DoS
LOCATION LeveragedToken.sol: 380
REVIEW Main Review

— DESCRIPTION

In the `_rebalance` function the streaming fee and rebalance fee is withdrawn from the margin of the system's account. However in cases where the streaming fee has grown to be significant and more than the available margin for the account this can result in a DoS of rebalances.

— RECOMMENDATION

This is an unlikely scenario however a permissioned function which can settle profits to the accounts margin can resolve this scenario if it were to ever arise.

— RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

Rebalances Can Occur While Paused

CATEGORY Warning
LOCATION LeveragedToken.sol
REVIEW Main Review

— DESCRIPTION

In the `LeveragedToken` contract the `rebalance` function can still be called by the keeper when the contract is in a paused state. This may lead to unexpected issues when the contract is paused.

— RECOMMENDATION

Be aware of this behavior and consider reverting in the `rebalance` function if the contract is paused.

— RESOLUTION

Status: Acknowledged.

LOW **L-19**

ACKNOWLEDGED

Order Fee Charged May Not Cover Total Fee

CATEGORY	Warning
LOCATION	LeveragedToken.sol
REVIEW	Main Review

— DESCRIPTION

The order fee charged in the `computePriceImpact` function is based on the `sizeDelta` of the individual deposit or withdrawal being made. As a result it is possible for the net amount of order fees collected from depositors/withdrawers to not cover the order fees levied by the Perps V3 system upon rebalancing.

This is because the `calculateOrderFee` function in Perps V3 charges a taker fee for the size delta which flips the skew. The taker fee is larger than the maker fee and so therefore many individual order fee computations will estimate that more volume will be charged at the maker fee, when in reality when the entire rebalance `sizeDelta` order is made more volume will be charged at the higher taker fee.

— RECOMMENDATION

Similar to L-03, order fees can be charged based on the incremental amount that the user contributes to the overall order fee that will be levied.

Given complications with the skew changing between the order fee being charged to withdrawers and depositors, the safest approach would be to charge users the entire order fee of the rebalance `sizeDelta` for maximum security against fee gaming. However there is a tradeoff with excessively charging users so these options should be weighed carefully.

— RESOLUTION

Status: Acknowledged.

LOW **L-20**

ACKNOWLEDGED

Leverage Not Reached Due To Pricing Discrepencies

CATEGORY Warning
LOCATION LeveragedToken.sol
REVIEW Main Review

DESCRIPTION

The LeveragedToken contract uses `_getLeverageUpdateSizeDelta` to calculate the size delta of the next order to maintain the intended leverage. Based on the current margin of the position, it calculates what should be the target size of the position: `uint256 targetNotional = marginAmount.mul(targetLeverage).div(assetPrice_);`

The issue is that it divided by `(assetPrice_);` rather than the exact price used by the PerpsV3 system, which may potentially differ if Perps uses a Chainlink feed for the market.

Consider the following example:

Margin: \$200

Intended Leverage: 2x

assetPrice_: \$100

SNX Price: \$110

$\text{targetNotional} = \$200 * 2 / \$100 = 4 \text{ tokens}$

This would return a `leverage()` of $4 * \$110 / \$200 = 2.2x$ which is higher than intended for the position.

RECOMMENDATION

Ensure the price feed used in PerpsV3 is also Pyth rather than Chainlink for each market a position is made, otherwise document this behavior.

RESOLUTION

Status: Acknowledged.

HIGH **H-01**

RESOLVED

Vault Drained By Cancellation Fees

CATEGORY Gaming
LOCATION Global
REVIEW Remediation Review

— DESCRIPTION

In the SNX perps market when a cancellation is performed the user who initiates the order cancellation is rewarded for invoking the transaction.

Orders can be submitted to the Perps V3 system that are immediately cancellable because the acceptablePrice is unfulfillable by the current market fill price. This is an acknowledged behavior from a previous Perps V3 audit referenced below.

The TLX vault can be placed into a state where every order committed is thereby immediately cancellable due to the skew exceeding the configured price impact tolerance.

While in this state, a malicious actor can drain the vault by intentionally minting small amounts such that an order is committed by the TLX system and then cancelling the order after the settlement delay has passed. For each cancellation the TLX vault will lose the keeper cancellation reward from it's margin and the cancelling keeper will gain it.

Reference finding from the SNX Perps V3 audit:
<https://app.notion.com/p/1ae2052b996749f1bbf9cf8f63ad33fd>

— RECOMMENDATION

Compare the current market fill price + a small buffer to the acceptable price. If the acceptable price is not met by the current market fill price + a small buffer then do not commit an order in the `_submitLeverageUpdate` function.

— RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/83>

MEDIUM **M-01**

ACKNOWLEDGED

Only Current Mint Amount Validated

CATEGORY Validation
LOCATION LeveragedToken.sol: 91
REVIEW Remediation Review

DESCRIPTION

In the `_validateMintAmount` function only the current amount being minted, represented as the `baseAmountIn` is validated against the max market size and value validations.

However several smaller mints could take place where each of the individual mint amounts remain below the max market validations, while the summation of the mints are above the max market validations. All of these mints may occur before a rebalance is triggered due to the somewhat significant deviation threshold of 25% of the margin value as a difference between OI and target OI.

RECOMMENDATION

Consider validating the current outstanding rebalance `sizeDelta` against the market maximums as opposed to the immediate `baseAmountIn` that is currently being minted.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Missing Buffer In Size Validation

CATEGORY Validation
LOCATION LeveragedToken.sol: 554
REVIEW Remediation Review

DESCRIPTION

In the `_validateMintAmount` function there is no buffer applied to the `maxMarketSize` like there is with the `maxSideValue` validation. As a result the vault's mint amount is more likely to drift over the `maxMarketSize` depending on the market price of the asset and the other interactions with the Perps V3 market outside of the TLX system.

RECOMMENDATION

Consider using a buffer to reduce the `maxMarketSize` to give some wiggle room for changing Perps market conditions.

RESOLUTION

Status: Resolved.

<https://github.com/TLX-Protocol/protocol-dev-v2/pull/84>

LOW **L-02**

ACKNOWLEDGED

Lacking safeTransfer

CATEGORY Best Practices
LOCATION LinkTopupUpkeep.sol
REVIEW Remediation Review

DESCRIPTION

In the `recoverAsset` function in the `LinkTopupUpkeep` contract transfer is used on the arbitrary `asset` token without validating the return value. This can lead to false positive successful execution of the `recoverAsset` function even when a token transfer fails for tokens that return false upon failure.

RECOMMENDATION

Consider using `safeTransfer` for the `recoverAsset` function.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Outstanding TODOs

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

In the `Addresses` and `Proxy0wner` contracts there are outstanding TODOs to update the missing addresses and to transfer ownership to the `Proxy0wner`.

— RECOMMENDATION

Be sure to address these outstanding TODO comments before deploying the system.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.